

Лабораторная работа №1. Быстрое погружение в язык Dart

Цель работы: Изучить основы языка **Dart**, сравнивая синтаксис и концепции с ранее изученными языками **Kotlin** и **C#**. Создать консольное приложение **ToDo**.

Необходимые инструменты:

- **Dart SDK 3.0+**
- **VS Code** (рекомендуется для лабораторной работы), либо **IntelliJ IDEA** / **Android Studio** с установленным плагином **Dart**
- **Git**

Теоретическая часть. Что такое Dart?

Dart — объектно-ориентированный язык программирования, разработанный **Google**.

Основные характеристики:

- **Статическая типизация** с выводом типов (type inference)
- **Sound Null Safety** — контроль nullable/nonnull значений на этапе компиляции
- **Встроенная асинхронность** (Future, async/await, Stream)
- Возможность компиляции:
 - в нативный код (AOT)
 - в JIT (для разработки)
 - в JavaScript (для Web)
- Основной язык фреймворка Flutter

Сравнение с знакомыми языками

Аспект	Dart	Kotlin	C#
Разработчик	Google	JetBrains	Microsoft
Платформа	Flutter, Web, Server	JVM, Native, JS	.NET
Null Safety	Обязательный	Обязательный	Опциональный (C# 8.0+)
Точка входа	<code>void main()</code>	<code>fun main()</code>	<code>static void Main()</code>
Вывод в консоль	<code>print()</code>	<code>println()</code>	<code>Console.WriteLine()</code>

Часть 1: Подготовка проекта

Подготовка проекта

1. Откройте ваш терминал (**Git Bash** / **PowerShell** / **Linux terminal**)
2. Проверьте что у вас установлен Dart (видео-шаги по установке есть на сайте):

dart --version

```
>$ dart --version
Dart SDK version: 3.11.0 (stable) (Mon Feb 9 00:38:07 2026 -0800) on "linux_x64"
```

Ожидаемый результат — отображение установленной версии SDK.

3. Перейдите в папку с вашими проектами (например **документами**):

cd Documents/

4. Создайте папку для лабораторных работ по **Flutter**:

mkdir Flutter/

cd Flutter

5. Создайте новый консольный проект:

dart create todo_app

Шаблон **console** создаёт минимальный CLI-проект. После создания появятся файлы:

```
Creating todo_app using template console...
.
.gitignore
analysis_options.yaml
CHANGELOG.md
pubspec.yaml
README.md
bin/todo_app.dart
lib/todo_app.dart
test/todo_app_test.dart

Running pub get... 2.0s
Resolving dependencies...
Downloading packages...
Changed 48 dependencies!

Created project todo_app in todo_app! In order to get started, run the following commands:

cd todo_app
dart run
```

6. Перейдите в папку проекта:

cd todo_app

7. Создайте папку для скриншотов:

mkdir img

8. Смотрим структуру:

ls -la

Структура проекта:

```
drwxr-xr-x  - denis 19 фев 10:20 .
drwxr-xr-x  - denis 19 фев 09:59 ..
drwxr-xr-x  - denis 19 фев 10:15 .dart_tool
drwxr-xr-x  - denis 19 фев 09:59 bin
drwxr-xr-x  - denis 19 фев 10:01 img
drwxr-xr-x  - denis 19 фев 09:59 lib
drwxr-xr-x  - denis 19 фев 09:59 test
-rw-r--r--  86 denis 19 фев 09:59 .gitignore
-rw-r--r-- 1,0k denis 19 фев 09:59 ! analysis_options.yaml
-rw-r--r--  29 denis 19 фев 09:59 CHANGELOG.md
-rw-r--r-- 11k denis 19 фев 09:59 . pubspec.lock
-rw-r--r-- 269 denis 19 фев 09:59 ! pubspec.yaml
-rw-r--r-- 122 denis 19 фев 09:59 README.md
```

- **dart_tool/** — Служебная папка (как **.gradle** в Android)
- **bin/** — Исполняемые файлы, точка входа (аналог **src/main**).

- **lib/** — Библиотеки/модули проекта.
- **test/** — Тесты
- **analysis_options.yaml** — Настройки анализатора кода
- **pubspec.yaml** — Зависимости. Аналог build.gradle.kts (Kotlin) или .csproj (C#)

Dart	Kotlin	C#	Python
pubspec.yaml	build.gradle	.csproj	pyproject.toml
pub.dev	Maven Central	NuGet	PyPI
dart pub get	gradle build	dotnet restore	pip install

Запуск проекта

1. Запуск стандартным способом:

dart run

Ожидаемый вывод:

```
└─$ dart run
Building package executable...
Built todo_app:todo_app.
Hello world: 42!
```

2. Можно также запустить конкретный файл:

```
└─$ dart run bin/todo_app.dart
Hello world: 42!
```

3. Также мы можем скомпилировать наш файл в нативный бинарник:

dart compile exe bin/todo_app.dart -o todo

```
└─$ dart compile exe bin/todo_app.dart -o todo
Generated: /home/denis/Project/Flutter/todo_app/todo
```

Мы можем убедиться что у нас появился файл командой **ls -la**:

```
.rw-r--r-- 6,6M denis 19 фев 10:21 todo
```

Важный момент

В моём случае файл создан в Linux:

- является **нативным ELF-бинарником**
- предназначен для текущей архитектуры

Он **не является кроссплатформенным**. Но я всё равно его могу запустить:

```
└─$ ./todo
Hello world: 42!
```

Важно понимать:

- бинарник Linux не запустится в Windows

- для Windows требуется компиляция в Windows

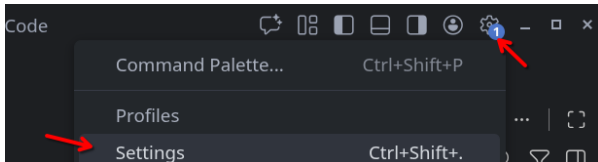
Проблема с отображением папки **bin**

1. Откройте проект в VS Code. Можно это сделать из терминала командой:

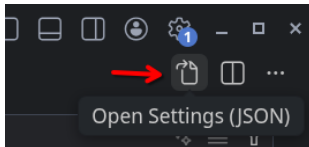
code .

2. По умолчанию у нас не будет видна папка **bin/** в **Explorer**. Это связано с тем, что мы прописывали конфигурацию для файлов **C#** в **settings.json**. Исправим это.

3. Откройте настройки (**Ctrl+,** на Windows):



4. Откройте **settings.json**:

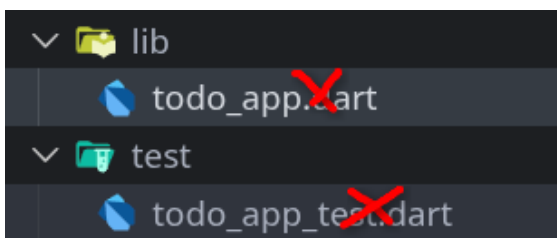


5. Найдите через поиск (**Ctrl+F**) настройки связанные с **bin**. Включим отображение папки:

```
"files.exclude": {  
  "**/bin": false,  
  "**/obj": true,  
}
```

Минимальная программа

1. Удалите файлы из папок **lib/** и **test/**:



2. Откройте файл **todo_app.dart**. Удалим его содержимое и заново напишем код:

```
void main() {  
  print('Hello world!');  
}
```

Запуск программы

В терминале введите:

dart run bin/todo_app.dart

Вывод:

```
>$ dart run bin/todo_app.dart
Hello world!
```

Инициализация Git-репозитория

1. Инициализируйте репозиторий:

```
git init -b main
```

2. Добавьте файлы в индекс:

```
git add .
```

3. Создайте первый коммит:

```
git commit -m "Initial dart project"
```

Подключение к GitHub

1. Создайте новый **public**-репозиторий на GitHub:

Flutter_Lab1

Без **README**, **.gitignore** и **License**.

2. Привяжите локальный репозиторий:

```
git remote add origin <URL_вашего_репозитория>
```

```
git push -u origin main
```



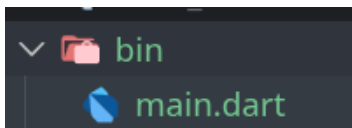
СКРИНШОТ 1: Сделайте скриншот терминала с выводом пуша. Сохраните в

img/step1_ВашаФамилия.png

Часть 2: Изучение языка Dart

Быстрый ввод в язык

1. В папке **bin/** создайте файл **main.dart**:



2. Откройте его и создайте в нём точку входа **main**:

```
void main() {
}
```

main() — стандартная точка входа программы.

- В отличие от C#, модификатор **static** не требуется.
- В отличие от Kotlin, **main** всегда возвращает **void**.

Весь демонстрационный код далее размещается **внутри main()** (если отдельно не указано иное). **Его нужно полностью переписать!**

2.1 Null Safety — ключевая особенность Dart

Dart использует **sound null safety** — переменная по умолчанию не может содержать **null**:

```
String name = 'Артём'; // не может быть null
String? name2 = null; // nullable тип
```

Принцип:

- **String** — гарантированно не **null**
- **String?** — допускает **null**

Аналогия:

- Kotlin: `String` / `String?`
- C# 8+: `string` / `string?`

Если попытаться присвоить **null** в **String**, компилятор выдаст ошибку.

2.2 Вывод типа — **var**

```
var count = 0; // int
var title = 'Учить Rust'; // String
```

var не означает "динамический тип".

Тип определяется во время компиляции и становится фиксированным.

Ошибка:

```
var x = 5;
x = 'text'; // ошибка компиляции
```

2.3 final — неизменяемая переменная

```
final id = 1;
```

- Значение присваивается один раз
- Известно во время выполнения
- Аналог `val` (Kotlin) или `readonly` (C#)

2.4 const — compile-time константа

```
const appName = 'TodoApp';
```

const:

- Значение известно на этапе компиляции
- Создается в compile-time

- Используется для неизменяемых объектов

Разница:

	final	const
Когда определяется	runtime	compile-time
Можно вычислять	да	нет
Используется для констант UI	редко	часто

2.5 Вывод в консоль

```
print('$name, $name2, $count, $title, $id, $appName');
```

Dart использует **string interpolation**:

- \$variable
- \${expression}

Пример:

```
${count + 5}
```

Настройка форматирования (VS Code)

Если строки автоматически переносятся, добавьте в **settings.json**:

```
"dart.lineLength": 70,
```

Рекомендуемая длина — 70–100 символов.

3. Типы данных

3.1 Коллекции (Generics)

Dart поддерживает параметризованные типы:

```
List<String> tags = ['институт', 'дом'];
Map<String, dynamic> data = {'key': 'value', 'num': 13};
Set<int> ids = {1, 2, 3};
```

List

Упорядоченная коллекция (как List в Kotlin и C#).

Map

Ассоциативный массив (ключ–значение).

Set

Множество уникальных элементов.

3.2 dynamic

```
dynamic anything = 42;
anything = 'now a string';
```

dynamic:

- Отключает проверку типов
- Проверки происходят во время выполнения
- Аналог object (C#) или Any без строгой проверки

Использование не рекомендуется в production-коде.

4. Функции

4.1 Стрелочная функция

```
String greet(String name) => 'Hello, $name!';
```

Короткая запись функции с одним выражением.

Аналог:

- **Kotlin:** fun greet(name: String) = "Hello \$name"
- **C#:** string Greet(string name) => ...

4.2 Named Parameters

Особенность Dart — именованные параметры.

```
void printTodo({required String title, bool done = false}) {  
  print('${done ? '+' : '-'} $title');  
}
```

Вызов:

```
printTodo(title: 'Купить молоко');  
printTodo(title: 'Проверить код', done: true);
```

Особенности:

- Параметры передаются по имени
- Можно задавать значения по умолчанию
- required — обязательный параметр

Именованные параметры активно используются во Flutter.

4.3 Positional Optional Parameters

```
String repeat(String s, [int times = 2]) => s * times;
```

Квадратные скобки означают:

- Параметр необязательный
- Имеет значение по умолчанию

5. Классы

Класс создаётся **вне main()**:

```
class Todo {  
  
}
```

5.1 Поля

Внутри класса добавляем поля:

```
final int id;  
String title;  
bool isDone;
```

- **final int id;** — неизменяемое поле
- **String title;** — изменяемое поле
- **bool isDone;** — состояние задачи

5.2 Конструктор с короткой записью

Внутри класса добавляем конструктор:

```
Todo({required this.id, required this.title, this.isDone = false});
```

this.id автоматически:

- создаёт поле
- присваивает значение

Аналог компактного конструктора в Kotlin.

5.3 Named Constructor

Внутри класса добавляем Named constructor:

```
Todo.empty() : id = 0, title = '', isDone = false;
```

В Dart нет перегрузки конструкторов как в C#.

Вместо этого используются **именованные конструкторы**.

5.4 Переопределение методов

Внутри класса переопределяем метод **toString()**:

```
@override  
String toString() => '${isDone ? '+' : '-'} [$id] $title';
```

Аннотация **@override**:

- Проверяется анализатором

- Предотвращает ошибки сигнатуры

Ключевые отличия от привычных языков

Особенность	Dart	Kotlin	C#
Null safety	String?	String?	string?
final / const	оба есть	val / const val	readonly / const
Named params	{required ...}	встроено	—
String template	'\$var \${expr}'	"\$var \${expr}"	"..."
Нет static class	нет	есть	есть
Точка входа	void main()	fun main()	static Main()



СКРИНШОТ 2: Сделайте скриншот класса `Todo`. Сохраните в

`img/step2_ВашаФамилия.png`

Выполните в терминале:

`git add .`

`git commit -m "learn dart"`

`git push`

Часть 3: Пишем TODO CLI

Создаём новый файл `lib/todo.dart`. В нём будет описана модель задачи.

Шаг 1. Пишем логику класса

Откройте файл `todo.dart` и создайте в нём класс `Todo`:

```
class Todo {
}
```

1.1. Статическое поле-счётчик

Внутри класса добавляем статический счётчик:

```
class Todo {
  +static int _counter = 0;
}
```

- `static` — поле принадлежит классу, а не экземпляру (как в Kotlin и C#)
- `_counter` — нижнее подчёркивание означает **приватность** в Dart (нет ключевого слова `private`)
- `int _counter = 0` — явная инициализация при объявлении

1.2. Поля экземпляра

Внутри класса добавляем поля:

```
final int id;  
String title;  
bool isDone;
```

- **final int id** — неизменяемое поле (аналог `val` в Kotlin, `readonly` в C#)
- **String title** — изменяемое строковое поле
- **bool isDone** — булево поле-состояние

1.3. Конструктор с короткой записью

Внутри класса добавляем конструктор:

```
Todo(this.title) : id = ++_counter, isDone = false;
```

- **Todo(this.title)** — короткая запись: `this.поле` автоматически создаёт поле и присваивает переданное значение. Аналог компактного конструктора в Kotlin (`data class`) или `primary constructor`.
- **: id = ++_counter, isDone = false** — **initializer list** (список инициализаторов). Выполняется до тела конструктора. Используется для инициализации `final`-полей.
- **++_counter** — сначала увеличивает счётчик, затем присваивает. Каждая задача получает уникальный `id`.

1.4. Переопределение метода `toString()`

Внутри класса переопределяем метод `toString()`:

```
@override  
String toString() {  
    String status;  
  
    if (isDone) {  
        status = "[x]";  
    } else {  
        status = "[ ]";  
    }  
  
    return "$status $id. $title";  
}
```

- **@override** — аннотация (как в Kotlin и C#). Проверяется анализатором, предотвращает ошибки сигнатуры.

- "\$status \$id. \$title" — строковая интерполяция (как в Kotlin "\$variable"). В C# аналог: \$"{status} {id}. {title}".
- Локальная переменная String status объявляется без инициализации — Dart допускает это, если гарантируется присвоение во всех ветках (if/else).

Выполните в терминале:

```
git add lib/todo.dart
```

```
git commit -m "Add Todo model"
```

```
git push
```

Шаг 2: Создаём файл lib/todo_repository.dart

Создаём новый файл `lib/todo_repository.dart`. В нём будет логика хранения и управления списком задач.

Импорт

В начало файла добавляем импорт:

```
import 'todo.dart';
```

- Dart использует **относительные пути** для импорта файлов внутри проекта.
- Аналог `using` в C# или `import` в Kotlin.

2.1. Класс и приватное поле-список

Внутри файла объявляем класс:

```
class TodoRepository {  
  final List<Todo> _todos = [];  
}
```

- `List<Todo>` — **обобщённый список** (аналог `List<Todo>` в C# и Kotlin).
- `[]` — литерал пустого списка. Аналог `listOf()` в Kotlin, `new List<Todo>()` в C#.
- `final + _todos` — список нельзя переназначить, но его содержимое менять можно.

2.2. Метод getAll()

Внутри класса добавляем метод `getAll()`:

```
List<Todo> getAll() {  
  return _todos;  
}
```

- Метод возвращает ссылку на внутренний список.
- Тип возвращаемого значения указывается перед именем метода (как в C# и Kotlin).

2.3. Метод `add()` валидация и исключение

Внутри класса добавляем метод `add()`:

```
void add(String title) {  
    if (title.trim().isEmpty) {  
        throw ArgumentError("Название задачи не может быть пустым");  
    }  
  
    Todo todo = Todo(title.trim());  
    _todos.add(todo);  
}
```

- `title.trim()` — обрезает пробелы по краям строки.
- `.isEmpty` — проверка на пустую строку (аналог `isEmpty()` в Kotlin, `string.IsNullOrEmpty` в C#).
- `throw ArgumentError(...)` — выброс исключения. В Dart нет checked exceptions — все исключения непроверяемые (как в Kotlin).
- `_todos.add(todo)` — добавление элемента в список.

2.4. Метод `complete()` — поиск по id через `for-in`

Внутри класса добавляем метод `complete()`:

```
void complete(int id) {  
    for (var todo in _todos) {  
        if (todo.id == id) {  
            todo.isDone = true;  
            return;  
        }  
    }  
  
    throw ArgumentError("Задача с id $id не найдена");  
}
```

- `for (var todo in _todos)` — цикл `for-in` (аналог `for (var todo in todos)` в C#, `for (todo in todos)` в Kotlin).
- `var` — вывод типа: Dart сам определяет, что `todo` — это `Todo`.
- Если задача найдена — изменяем поле и выходим через `return`.
- Если не найдена — выбрасываем исключение после цикла.

2.5. Метод delete() — поиск по индексу через for с индексом

Внутри класса добавляем метод `delete()`:

```
void delete(int id) {  
    for (int i = 0; i < _todos.length; i++) {  
        if (_todos[i].id == id) {  
            _todos.removeAt(i);  
            return;  
        }  
    }  
  
    throw ArgumentError("Задача с id $id не найдена");  
}
```

- Классический `for` с индексом — нужен, чтобы удалить элемент по позиции через `removeAt(i)`.
- `_todos.length` — количество элементов (аналог `.size` в Kotlin, `.Count` в C#).
- `_todos[i]` — обращение по индексу (как в обоих языках).
- `removeAt(i)` — удаление по индексу из списка.

При удалении **нельзя использовать** `for-in` и модифицировать список одновременно — это вызовет ошибку во время выполнения.

Выполните в терминале:

`git add lib/todo_repository.dart`

`git commit -m "Add TodoRepository Фамилия"` (напишите вашу фамилию)

`git push`



СКРИНШОТ 3: Сделайте скриншот коммита и пуша. Сохраните в `img/step3_ВашаФамилия.png`

Шаг 3: Заменяем содержимое `bin/todo_app.dart`

Это точка входа в приложение. Открываем уже существующий файл `bin/todo_app.dart` и заменяем его содержимое.

3.1. Импорты и функция `main()`

Добавляем импорты:

```
import 'dart:io';  
import 'package:todo_app/todo_repository.dart';  
import 'package:todo_app/todo.dart';
```

- **import 'dart:io'** — встроенная библиотека для работы с вводом/выводом (консоль, файлы).
- **import 'package:todo_app/...'** — импорт из пакета по имени проекта (настраивается в pubspec.yaml).

Пишем функцию **main()**:

```
void main() {
  TodoRepository repo = TodoRepository();
  printMenu();
  while (true) {
    stdout.write("> ");
    String? input = stdin.readLineSync();
    if (input == null) {
      continue;
    }
    input = input.trim();
    if (input.isEmpty) {
      continue;
    }
    bool shouldExit = handleCommand(repo, input);
    if (shouldExit) {
      break;
    }
  }
}
```

- **stdout.write("> ")** — вывод без перевода строки (аналог print без \n; в C# — Console.Write).
- **String? input** — **nullable** тип. ? означает, что переменная может быть null. В Kotlin: String?, в C#: string?.
- **stdin.readLineSync()** — чтение строки из консоли. Возвращает String? (может вернуть null при EOF).
- **while (true) + break** — бесконечный цикл с явным выходом (стандартная идиома для CLI).
- **continue** — пропустить итерацию и перейти к следующей.

У нас пока что нету методов **printMenu()** и **handleCommand()**, поэтому они подсвечиваются.

3.2. Функция printMenu()

Внутри файла `todo_app.dart`, создадим нужные функции, для работы нашего **Todo**.

Добавим функцию `printMenu()`:

```
void printMenu() {  
  print("Консольное приложение TODO");  
  print("Команды:");  
  print(" add <текст>      - добавить задачу");  
  print(" list              - показать список");  
  print(" done <id>          - отметить выполненной");  
  print(" delete <id>         - удалить задачу");  
  print(" exit               - выход");  
  print("");  
}
```

- Функции в Dart объявляются **вне классов** (как в Kotlin, в отличие от C# где нужен класс).
- **void** — функция ничего не возвращает.
- **print()** — вывод строки с переводом строки (аналог `println` в Kotlin, `Console.WriteLine` в C#).

3.3. Функции-обработчики команд

Добавим функцию `addCommand()` — разбор текста задачи из строки ввода:

```
void addCommand(TodoRepository repo, String input) {  
  if (input.length <= 4) {  
    print("Ошибка: введите текст задачи");  
    return;  
  }  
  String title = input.substring(4).trim();  
  repo.add(title);  
  print("Задача добавлена");  
}
```

- **input.length** — длина строки.
- **input.substring(4)** — строка начиная с индекса 4, т.е. текст после "add".
- Ранний **return** при ошибке — паттерн "guard clause".

Добавим функцию `listCommand()` — вывод списка с проверкой на пустоту:


```
void listCommand(TodoRepository repo) {
    List<Todo> todos = repo.getAll();
    if (todos.isEmpty) {
        print("Список задач пуст");
        return;
    }
    for (var todo in todos) {
        print(todo);
    }
}
```

- `print(todo)` — автоматически вызывает `todo.toString()`, который мы переопределили.

Добавим функции `doneCommand()` и `deleteCommand()` — парсинг id:

```
void doneCommand(TodoRepository repo, List<String> parts) {
    if (parts.length < 2) {
        print("Ошибка: укажите id");
        return;
    }
    int id = int.parse(parts[1]);
    repo.complete(id);
    print("Задача отмечена выполненной");
}
```

```
void deleteCommand(TodoRepository repo, List<String> parts) {
    if (parts.length < 2) {
        print("Ошибка: укажите id");
        return;
    }
    int id = int.parse(parts[1]);
    repo.delete(id);
    print("Задача удалена");
}
```

- `List<String> parts` — список частей строки, разделённых пробелом.
- `int.parse(parts[1])` — преобразование строки в целое число (аналог `toInt()` в Kotlin, `int.Parse()` в C#).

3.4. Функция `handleCommand()`

Добавим функцию `handleCommand()`:

```

bool handleCommand(TodoRepository repo, String input) {
  List<String> parts = input.split(" ");
  String command = parts[0].toLowerCase();
  try {
    switch (command) {
      case "add":
        addCommand(repo, input);
        break;
      case "list":
        listCommand(repo);
        break;
      case "done":
        doneCommand(repo, parts);
        break;
      case "delete":
        deleteCommand(repo, parts);
        break;
      case "exit":
        print("Выход из программы");
        return true;
      default:
        print("Неизвестная команда");
    }
  } catch (e) {
    print("Ошибка: $e");
  }
  return false;
}

```

- **input.split(" ")** — разбивка строки по пробелу в список (аналог `split(" ")` в Kotlin, `Split(' ')` в C#).
- **.toLowerCase()** — приведение к нижнему регистру.
- **switch/case** — требует `break` в каждой ветке (в отличие от Kotlin `when`, как в C#).
- **default** — ветка по умолчанию (аналог `else` в `when / default` в C#).
- **try { } catch (e) { }** — перехват любого исключения. В Dart `catch (e)` без указания типа перехватывает всё. Аналог `catch (Exception e)` в C#.
- Функция возвращает **bool** — `true` означает выход из цикла.

Выполните в терминале:

git add bin/todo_app.dart

git commit -m "Add main entry point with CLI loop - Фамилия"

git push



СКРИНШОТ 4: Сделайте скриншот работающего приложения в терминале, а также коммит + пуш. Сохраните в **img/step4_ВашаФамилия.png**

Запуск приложения

В терминале введите:

dart run bin/todo_app.dart

Проверьте работоспособность приложения. Пример сессии:

```
└─$ dart run bin/todo_app.dart
Консольное приложение TODO
Команды:
  add <текст>      - добавить задачу
  list             - показать список
  done <id>        - отметить выполненной
  delete <id>      - удалить задачу
  exit             - выход

> add Купить продукты
Задача добавлена
> add Сделать дз по РМП
Задача добавлена
> list
[ ] 1. Купить продукты
[ ] 2. Сделать дз по РМП
> done 1
Задача отмечена выполненной
> list
[x] 1. Купить продукты
[ ] 2. Сделать дз по РМП
> delete 2
Задача удалена
> list
[x] 1. Купить продукты
> exit
Выход из программы
```

Часть 4: Пакеты и pub.dev

pubspec.yaml

В корне проекта откройте **pubspec.yaml**.

Найдите раздел **dependencies**. Обратите внимание как после выполнения действий сюда подключится пакет.

Введите в терминале:

dart pub add ansicolor

dart pub get

Вывод в терминале:

```
└─>$ dart pub add ansicolor
Resolving dependencies...
Downloading packages...
+ ansicolor 2.0.3
Changed 1 dependency!
└─[denis@denis-Desktop:~/P/F
└─>$ dart pub get
Resolving dependencies...
Downloading packages...
Got dependencies!
```

Dart обращается к **pub.dev**

- Загружает пакет
- Обновляет pubspec.lock

Использование пакета

Откройте **bin/todo_app.dart**. Добавьте импорт:

```
import 'package:ansicolor/ansicolor.dart';
```

Добавим глобальные переменные (выше **main()**):

```
final AnsiPen greenPen = AnsiPen()..green();
final AnsiPen redPen = AnsiPen()..red();
final AnsiPen bluePen = AnsiPen()..blue();
final AnsiPen yellowPen = AnsiPen()..yellow();
```

Каскадный оператор .. позволяет вызывать методы на объекте без повторного указания имени

Изменим вывод

Пример: меню

```
void printMenu() {
  print(yellowPen("Консольное приложение TODO"));
}
```

Пример: ошибки

В **handleCommand()** замените:

```
print(redPen("Ошибка: $e"));
```

Аналогично примените для других элементов и поэкспериментируйте с цветами (например добавление задач сделать синим, успешные действия добавления задачи зелёным и т. д.).

Где искать пакеты

- **pub.dev** — официальный реестр
- Метрики: Likes, Pub Points, Popularity
- Фильтр: Flutter / Dart, Platform support

Популярные пакеты — пригодятся при изучении Flutter

Пакет	Назначение
http	HTTP запросы
dio	Продвинутый HTTP клиент
json_serializable	Генерация JSON кода
freezed	Иммутабельные классы (аналог data class)
riverpod	State management
drift	Локальная БД (SQLite)

 **СКРИНШОТ 5:** Откройте pub.dev (сайт) в браузере, найдите пакет **ansicolor**, сделайте скриншот страницы пакета с метриками. Сохраните в **img/step5_ВашаФамилия.png**

Выполните в терминале:

git add .

git commit -m "add package ansicolor"

git push

Часть 5: Async/Await

Dart использует **один поток выполнения** с механизмом **event loop**, аналогично JavaScript. Это означает:

- Код выполняется последовательно.
- Долгие операции (ожидание сети, файла, таймера) не блокируют программу.
- Вместо блокировки используется Future и async/await.

Асинхронность — встроенная часть языка.

1. Создайте и откройте новый файл **bin/async.dart**

2. В начале файла добавьте импорт:

```
import 'dart:io';
```

- `dart:io` — стандартная библиотека для:
- ввода/вывода
- работы с файлами
- сетевых операций

3. Создайте объект **Future<T>**:

```
Future<String> readUserInput() async {  
  print('Enter something:');  
  return stdin.readLineSync() ?? '';  
}
```

`Future<T>` — это объект, который:

- сейчас не содержит результат
- получит его позже

Почему функция `async`?

Ключевое слово **`async`**:

- позволяет использовать **`await`**
- автоматически оборачивает возвращаемое значение в `Future`

Даже если внутри нет **`await`**, функция всё равно возвращает `Future`.

4. Создадим имитацию **HTTP-запроса**:

```
Future<List<String>> fetchTodos() async {  
  await Future.delayed(Duration(seconds: 1)); // ждём 1 сек  
  return ['Remote todo 1', 'Remote todo 2'];  
}
```

- `Future.delayed`

Создаёт `Future`, который завершится через указанное время.

- `Duration(seconds: 1)`

Создаёт объект времени.

- `await`

Ключевое слово **`await`**:

- приостанавливает выполнение текущей функции
- не блокирует поток

- позволяет другим событиям выполняться

Важно: Это **не Thread.Sleep**. Это неблокирующее ожидание.

5. Асинхронный main:

В Dart main может быть async.

```
void main() async {  
}
```

Внутри **main** допишем:

```
final input = await readUserInput();  
print('You entered: $input');  
print('Loading todos...');  
final todos = await fetchTodos();  
todos.forEach(print);
```

Запустите выполнение программы:

dart bin/async.dart

Ожидаемый вывод:

```
↳$ dart bin/async.dart  
Enter something:  
32434  
You entered: 32434  
Loading todos...  
Remote todo 1  
Remote todo 2
```

Пошаговое выполнение программы:

1. Запускается main
2. Вызывается readUserInput()
3. await ждёт завершения Future
4. Пользователь вводит строку
5. Печатается результат
6. Вызывается fetchTodos()
7. await приостанавливает функцию на 1 секунду
8. После таймера возвращается список
9. Список выводится в консоль



СКРИНШОТ 6: Сделайте скриншот кода **async.dart** и вывода в терминале.

Сохраните в **img/step6_ВашаФамилия.png**

Выполните в терминале:

git add .

git commit -m "docs: add async examples to bin"

git push

Самостоятельное задание. Оформление README.md

Что должно быть в README

Заголовок и описание: Название проекта и одно-два предложения о чём он.

Информация об авторе: Ваше имя, группа.

Скриншот приложения: Вставьте скриншот вашего работающего приложения.

Как запустить: Опишите шаги, чтобы любой человек мог клонировать репозиторий и запустить проект:

Что изучили: Коротко — 3–5 пунктов что нового узнали в этой лабораторной.

7. Ответьте на вопросы:

1. Чем **final** отличается от **const** в Dart?
2. Что означает **String**?
3. Чем Future отличается от обычного значения? Что означает await с точки зрения потока выполнения?
4. Зачем в Dart именованные конструкторы, если в C# есть перегрузка?

Сделайте коммит:

git add .

git commit -m "added README description and completed labs"

git push

Полезные ссылки

- dart.dev/learn — официальный туториал
- dartpad.dev — онлайн редактор
- pub.dev — пакеты
- dart.dev/language — справочник по языку
- [Effective Dart](https://effective.dart.dev) — гайд по стилю кода